AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 10: Optimisation

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Gradient descent, Newton's method, linear, super-linear and quadratic convergence, condition number, BFGS, Levenberg–Marquardt

2 Main Points

2.1 Machine Learning as Optimisation

- Once you've designed your learning machine and chosen your loss function the rest is optimisation: find the weights $\boldsymbol{w}$ that minimise the loss $L(\boldsymbol{w})$
- This lecture covers the classical methods, which work well for up to thousands of parameters (L-BFGS: millions) when we can afford gradients computed from all the data. The next lecture covers the very large problems of deep learning

2.2 Gradient Descent

- At a minimum (or a maximum, or a saddle point) $\boldsymbol{\nabla} L = \mathbf{0}$. For most models we cannot solve this, so we search iteratively
- The gradient points in the direction in which the loss increases fastest, so we step the other way
$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - r\,\boldsymbol{\nabla} L(\boldsymbol{w}^{(t)})$$
- The learning rate (step size) r is the difficulty: too small and we crawl, too large and we diverge

2.3 Newton's Method

- Near a minimum almost every loss is quadratic (higher order terms in the Taylor expansion are negligible). In one dimension, if $f(x) = a + \frac{b}{2}(x - x^*)^2$ then $f'(x) = b\,(x - x^*)$ and $f''(x) = b$, so
$$x^* = x - \frac{f'(x)}{f''(x)}$$
- In many dimensions the second derivative becomes the Hessian, $H_{ij} = \partial^2 L/\partial w_i\,\partial w_j$, and
$$\boldsymbol{w}^* = \boldsymbol{w} - \mathbf{H}^{-1}\boldsymbol{\nabla} L(\boldsymbol{w})$$
- For a loss that is not exactly quadratic we iterate this. Close to a minimum it converges *quadratically*: an error ϵ becomes one of order ϵ^2, then ϵ^4, and so on, roughly doubling the number of correct digits each step (Exercise 4)

- Its drawbacks: the Hessian has N^2 entries and is costly to compute and invert, and far from a minimum Newton's method can head in the wrong direction (towards a maximum or saddle, or anywhere)

2.4 Step Size and Different Length Scales

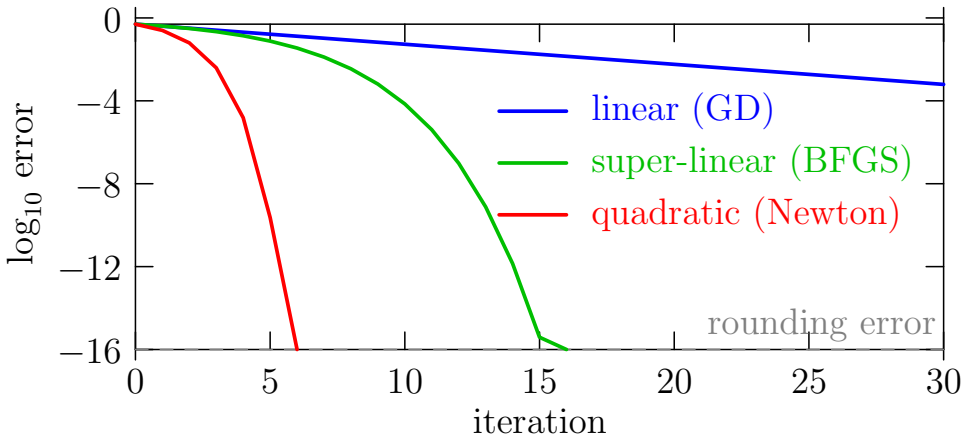
- Near a minimum $\boldsymbol{w}^*$ take the quadratic approximation $L(\boldsymbol{w}) \approx L(\boldsymbol{w}^*) + \frac{1}{2}(\boldsymbol{w}-\boldsymbol{w}^*)^\mathsf{T}\mathbf{H}(\boldsymbol{w}-\boldsymbol{w}^*)$. A gradient step gives
$$\boldsymbol{w}^{(t+1)} - \boldsymbol{w}^* = (\mathbf{I} - r\,\mathbf{H})\,(\boldsymbol{w}^{(t)} - \boldsymbol{w}^*)$$

- Write $\mathbf{H} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}$ and $u_i = \boldsymbol{v}_i^\mathsf{T}(\boldsymbol{w} - \boldsymbol{w}^*)$, the error along the i-th eigenvector. Then each component evolves on its own:
$$u_i^{(t+1)} = (1 - r\,\lambda_i)\,u_i^{(t)}$$

- Every component must shrink, $|1 - r\lambda_i| < 1$, so we need $r < 2/\lambda_{\max}$. The steepest direction limits the step size

- With r that small, the flattest direction shrinks by $1 - r\lambda_{\min} > 1 - 2\lambda_{\min}/\lambda_{\max}$ per step. The best fixed step, $r = 2/(\lambda_{\max} + \lambda_{\min})$, gives the factor
$$\frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$$
where κ is the condition number of the Hessian (eigensystems notes). Badly conditioned problems ($\kappa \gg 1$) are slow

- **Rates of convergence.** If the error shrinks by a constant factor each step, convergence is *linear* (a straight line on a log plot). If the factor itself tends to zero it is *super-linear*, and if the error is squared each step it is *quadratic*



Gradient descent is linear, quasi-Newton methods such as BFGS are super-linear, and Newton's method is quadratic

2.5 Line Minimisation and Zig-Zags

- Rather than a fixed step we can minimise along the gradient direction (a line search) at every step

- At the line minimum the new gradient is orthogonal to the direction we came from, so in a narrow valley successive steps are at right angles: the path zig-zags across the valley rather than going along it

2.6 Quasi-Newton Methods

- These use curvature information without computing the full Hessian
- **Conjugate gradient** chooses search directions that do not undo the progress made along earlier ones. With exact line searches it reaches the minimum of an N-dimensional quadratic in N steps
- **BFGS** (Broyden–Fletcher–Goldfarb–Shanno) builds an approximation $\mathbf{B} \approx \mathbf{H}^{-1}$ from the gradients it has already seen, requiring at each step that

  $$\mathbf{B}\left(\boldsymbol{\nabla} L(\boldsymbol{w}^{(t+1)}) - \boldsymbol{\nabla} L(\boldsymbol{w}^{(t)})\right) = \boldsymbol{w}^{(t+1)} - \boldsymbol{w}^{(t)}$$

  (for a quadratic, $\boldsymbol{\nabla} L$ changes by exactly $\mathbf{H}$ times the step). It steps along $-\mathbf{B}\,\boldsymbol{\nabla} L$ with a line search and converges super-linearly
  - $\mathbf{B}$ is $N \times N$. *L-BFGS* keeps only the last ten or so steps and gradient changes, so its memory is $O(N)$
- **Levenberg–Marquardt** is for least-squares losses $L(\boldsymbol{w}) = \|\boldsymbol{\epsilon}(\boldsymbol{w})\|^2$ with residuals $\epsilon_i = f(\boldsymbol{x}_i|\boldsymbol{w}) - y_i$
  - Linearise the residuals about the current weights, $\boldsymbol{\epsilon}(\boldsymbol{w}) \approx \boldsymbol{\epsilon}(\boldsymbol{w}^{(k)}) + \mathbf{J}(\boldsymbol{w} - \boldsymbol{w}^{(k)})$, where $\mathbf{J}$ is the Jacobian $J_{ij} = \partial\epsilon_i/\partial w_j$
  - Minimising the linearised loss gives the Gauss–Newton step $\boldsymbol{w}^{(k+1)} = \boldsymbol{w}^{(k)} - (\mathbf{J}^\mathsf{T}\mathbf{J})^{-1}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)})$: Newton's method with $\mathbf{J}^\mathsf{T}\mathbf{J}$ standing in for the Hessian
  - Adding a penalty $\nu \left\|\boldsymbol{w} - \boldsymbol{w}^{(k)}\right\|^2$ on the step size gives the Levenberg–Marquardt step (Exercise 5)

    $$\boldsymbol{w}^{(k+1)} = \boldsymbol{w}^{(k)} - (\mathbf{J}^\mathsf{T}\mathbf{J} + \nu\,\mathbf{I})^{-1}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)})$$

  - Large ν gives a short step along the negative gradient; small ν gives Gauss–Newton. ν is reduced when the linear model predicts the drop in loss well and increased when it does not (a *trust region*)
- Non-gradient methods (Nelder–Mead, Powell's method, evolutionary strategies) also exist, but are not competitive in high dimensions

2.7 Where These Methods Are Used

- Logistic regression: scikit-learn's default solver is L-BFGS, and Newton's method is the standard fit for generalised linear models
- Gaussian processes: the kernel hyper-parameters are usually fitted with L-BFGS
- Curve fitting (non-linear least squares): Levenberg–Marquardt, e.g. SciPy's `curve_fit`
- Support vector machines: training is a quadratic programme (later lectures)
- With millions to billions of parameters, and gradients estimated from mini-batches, we need the methods of the next lecture

3 Exercises

1. **Divergence.** Take $L(w) = \frac{1}{2}w^2$ (one dimension) and update $w^{(t+1)} = w^{(t)} - r\,L'(w^{(t)})$
   (a) Compute the optimum step size
   (b) What is $r_{\max}$ such that we no longer converge if $r \geq r_{\max}$?

(c) If $r > r_{\max}$ calculate how $w^{(t)}$ grows with time

2. **Quadratic optima.** Consider the 2-d loss $L(\boldsymbol{w}) = w_1^2/2 + w_2^2 - w_1\, w_2$

   (a) Compute the gradient

   (b) Compute the Hessian

   (c) Compute the eigenvalues of the Hessian

   (d) Plot the contour lines of $L(\boldsymbol{w})$

3. **Condition number.** For the loss in Exercise 2

   (a) Find the condition number κ and the largest stable step size

   (b) With the best fixed step $r = 2/(\lambda_{\max} + \lambda_{\min})$, how many gradient steps reduce the error by a factor 10^6?

   (c) How many steps does Newton's method need?

4. **Newton's method converges quadratically.** Let $\epsilon = x - x^*$ be the error before a Newton step and ϵ' the error after. By Taylor expanding $f'(x)$ and $f''(x)$ about x^*, show that

$$\epsilon' \approx \frac{f'''(x^*)}{2\, f''(x^*)}\, \epsilon^2$$

5. **Levenberg–Marquardt.** Show that minimising $\|\boldsymbol{\epsilon} + \mathbf{J}\,\boldsymbol{d}\|^2 + \nu\, \|\boldsymbol{d}\|^2$ over the step $\boldsymbol{d}$ gives $\boldsymbol{d} = -(\mathbf{J}^\mathsf{T}\mathbf{J} + \nu\, \mathbf{I})^{-1}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}$. What happens when ν is very large?

4 Experiments

4.1 Comparing optimisers on the Rosenbrock function

- The Rosenbrock function $L = 100\,(w_2 - w_1^2)^2 + (1 - w_1)^2$ (the example in the lecture, with residuals $\epsilon_1 = 10\,(w_2 - w_1^2)$ and $\epsilon_2 = 1 - w_1$) has a long, curved, narrow valley

```
import numpy as np
from scipy.optimize import minimize, least_squares, rosen, rosen_der, rosen_hess

x0 = np.array([-1.2, 1.0])        # the classic start; the minimum is at (1, 1)

x, r = x0.copy(), 1e-3            # gradient descent: a larger r diverges
for t in range(1, 200001):
    x = x - r * rosen_der(x)
    if np.linalg.norm(x - 1) < 1e-6: break
print("gradient descent:", t, "steps")

for method in ["BFGS", "L-BFGS-B", "trust-exact"]:   # trust-exact: Newton
    hess = rosen_hess if method == "trust-exact" else None
    res = minimize(rosen, x0, method=method, jac=rosen_der, hess=hess, tol=1e-12)
    print(f"{method}: {res.nit} iterations")

# Levenberg-Marquardt on the residuals from the lecture
eps = lambda x: np.array([10 * (x[1] - x[0]**2), 1 - x[0]])
res = least_squares(eps, x0, method="lm", xtol=1e-12)
print("Levenberg-Marquardt:", res.nfev, "function evaluations")
```

- Gradient descent needs about 34,000 steps; BFGS, L-BFGS, Newton (with a trust region) and Levenberg–Marquardt need 20–40. Try increasing `r` to see gradient descent diverge

5 Answers

1. The update is $w^{(t+1)} = (1 - r)\, w^{(t)}$

   (a) The optimum step size is $r = 1$, which jumps straight to the minimum at $w = 0$

   (b) $r_{\max} = 2$, where $|1 - r| = 1$

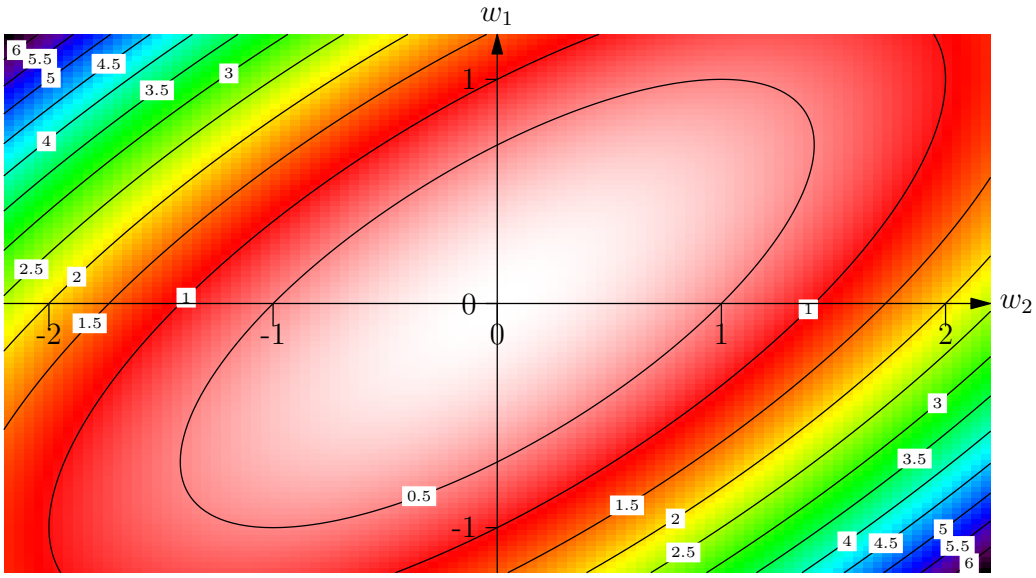
   (c) $w^{(t)} = (1 - r)^t\, w^{(0)}$: its size grows as $(r - 1)^t$, with alternating sign

2. Quadratic optima

   (a)
   $$\nabla L(\boldsymbol{w}) = \begin{pmatrix} w_1 - w_2 \\ 2\, w_2 - w_1 \end{pmatrix}$$

   (b)
   $$\mathbf{H} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix}$$

   (c) Let $T = \operatorname{tr} \mathbf{H} = 3$ and $D = \det(\mathbf{H}) = 1$ then $\lambda = \frac{1}{2}\left(T \pm \sqrt{T^2 - 4\,D}\right) = (3 \pm \sqrt{5})/2 = \{0.382, 2.618\}$

   (d)
   

3. Condition number

   (a) $\kappa = 2.618/0.382 \approx 6.85$, and stability needs $r < 2/2.618 \approx 0.76$

   (b) $r = 2/3$ gives the factor $(\kappa - 1)/(\kappa + 1) \approx 0.745$ per step, so we need n with $0.745^n = 10^{-6}$: $n = \ln 10^{-6} / \ln 0.745 \approx 47$ steps

   (c) One: the loss is exactly quadratic, so a single Newton step lands on the minimum

4. Expanding about x^*, where $f'(x^*) = 0$: $f'(x) \approx f''\epsilon + \frac{1}{2} f'''\epsilon^2$ and $f''(x) \approx f'' + f'''\epsilon$ (derivatives at x^*). Then
   $$\epsilon' = \epsilon - \frac{f'(x)}{f''(x)} \approx \epsilon - \epsilon\, \frac{1 + \frac{f'''}{2f''}\epsilon}{1 + \frac{f'''}{f''}\epsilon} \approx \epsilon - \epsilon\left(1 - \frac{f'''}{2f''}\epsilon\right) = \frac{f'''}{2f''}\,\epsilon^2$$
   using $1/(1 + a) \approx 1 - a$ for small a

5. The gradient with respect to $\boldsymbol{d}$ is $2\mathbf{J}^\mathsf{T}(\boldsymbol{\epsilon} + \mathbf{J}\boldsymbol{d}) + 2\nu\, \boldsymbol{d}$. Setting it to zero gives $(\mathbf{J}^\mathsf{T}\mathbf{J} + \nu\, \mathbf{I})\, \boldsymbol{d} = -\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}$. For large ν, $\boldsymbol{d} \approx -\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}/\nu = -\nabla L/(2\nu)$: a short gradient-descent step